

Advanced Sorts: Merge Sort & Quick Sort

April 14, 2026

Introduction: Moving Beyond Elementary Sorts

Elementary sorts (Selection and Insertion Sort) are easy to understand but run in $O(n^2)$ time, which becomes unacceptably slow once n grows into the thousands. This tutorial introduces two *divide-and-conquer* sorting algorithms that achieve the theoretically optimal $O(n \log n)$ average-case performance:

1. **Merge Sort** — divide the array in half, sort each half recursively, then merge the two sorted halves.
2. **Quick Sort** — pick a pivot, partition the array around it, then recurse on each partition.
3. **Time Complexity Analysis** — best, average, and worst case for both, with empirical verification.

Part 1: Merge Sort

1.1 Core Idea**How Merge Sort Works**

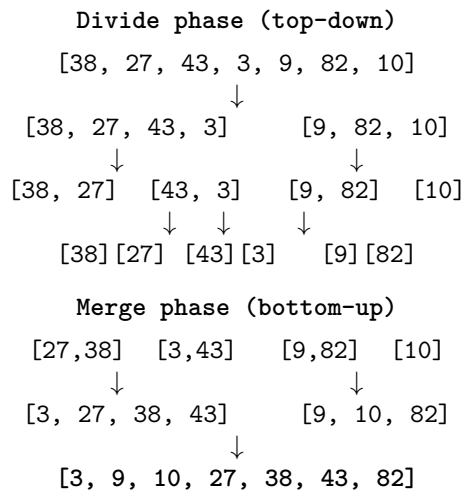
Merge Sort follows the classic **Divide-and-Conquer** strategy in three phases:

1. **Divide** — split the array at the midpoint into a left half and a right half.
2. **Conquer** — recursively sort each half (base case: an array of size 1 is already sorted).
3. **Merge** — combine the two sorted halves into a single sorted array by repeatedly picking the smaller front element.

The key insight is that **merging two sorted arrays** is an $O(n)$ operation. Because the array is halved at every level, there are only $O(\log n)$ levels, giving a total of $O(n \log n)$ work.

1.2 Recursion Tree

The diagram below shows how Merge Sort breaks down and rebuilds the array [38, 27, 43, 3, 9, 82, 10]:



1.3 Step-by-Step Trace: Merge Operation

The core of Merge Sort is the **merge** step. Here is a detailed trace merging [3, 27, 38, 43] and [9, 10, 82]:

Left ptr	Right ptr	Output so far	Action
3	9	[]	$3 < 9 \rightarrow$ take 3 from left
27	9	[3]	$27 > 9 \rightarrow$ take 9 from right
27	10	[3, 9]	$27 > 10 \rightarrow$ take 10 from right
27	82	[3, 9, 10]	$27 < 82 \rightarrow$ take 27 from left
38	82	[3, 9, 10, 27]	$38 < 82 \rightarrow$ take 38 from left
43	82	[3, 9, 10, 27, 38]	$43 < 82 \rightarrow$ take 43 from left
--	82	[3, 9, 10, 27, 38, 43]	left exhausted $\rightarrow$ append remaining right
[3, 9, 10, 27, 38, 43, 82]			Merged result

1.4 Implementation

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Merge two sorted halves: arr[left..mid] and arr[mid+1..right]
6 void merge(vector<int>& arr, int left, int mid, int right) {
7     // Copy both halves into temporary arrays
8     vector<int> L(arr.begin() + left, arr.begin() + mid + 1);
9     vector<int> R(arr.begin() + mid + 1, arr.begin() + right + 1);
10
11     int i = 0, j = 0, k = left;
12
13     // Pick the smaller element from L or R each time
14     while (i < (int)L.size() && j < (int)R.size()) {
15         if (L[i] <= R[j]) // <= keeps the sort stable
16             arr[k++] = L[i++];
17         else
18             arr[k++] = R[j++];
19     }
20

```

```

21 // Append any remaining elements
22 while (i < (int)L.size()) arr[k++] = L[i++];
23 while (j < (int)R.size()) arr[k++] = R[j++];
24 }
25
26 // Recursive Merge Sort
27 void mergeSort(vector<int>& arr, int left, int right) {
28     if (left >= right) return; // Base case: 1 element
29
30     int mid = left + (right - left) / 2; // Avoid integer overflow
31     mergeSort(arr, left, mid); // Sort left half
32     mergeSort(arr, mid + 1, right); // Sort right half
33     merge(arr, left, mid, right); // Merge sorted halves
34 }
35
36 void printArray(const string& label, const vector<int>& arr) {
37     cout << label << ": [ ";
38     for (int x : arr) cout << x << " ";
39     cout << "]\n";
40 }
41
42 int main() {
43     vector<int> data = {38, 27, 43, 3, 9, 82, 10};
44     printArray("Before", data);
45     mergeSort(data, 0, data.size() - 1);
46     printArray("After ", data);
47 }

```

Listing 1: Merge Sort — complete implementation with trace output

Expected output:

Before: [38 27 43 3 9 82 10]
 After : [3 9 10 27 38 43 82]

1.5 Generic Template Version

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 using namespace std;
5
6 template <typename T>
7 void merge(vector<T>& arr, int left, int mid, int right) {
8     vector<T> L(arr.begin() + left, arr.begin() + mid + 1);
9     vector<T> R(arr.begin() + mid + 1, arr.begin() + right + 1);
10    int i = 0, j = 0, k = left;
11    while (i < (int)L.size() && j < (int)R.size())
12        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
13    while (i < (int)L.size()) arr[k++] = L[i++];
14    while (j < (int)R.size()) arr[k++] = R[j++];
15 }
16
17 template <typename T>
18 void mergeSort(vector<T>& arr, int left, int right) {
19     if (left >= right) return;
20     int mid = left + (right - left) / 2;
21     mergeSort(arr, left, mid);

```

```

22     mergeSort(arr, mid + 1, right);
23     merge(arr, left, mid, right);
24 }
25
26 template <typename T>
27 void print(const vector<T>& arr) {
28     for (const T& x : arr) cout << x << " ";
29     cout << "\n";
30 }
31
32 int main() {
33     vector<int>    ints    = {5, 2, 8, 1, 9, 3};
34     vector<double> dbls    = {3.14, 1.41, 2.72, 0.57};
35     vector<string> words   = {"banana", "apple", "cherry", "date"};
36
37     mergeSort(ints, 0, ints.size() - 1); print(ints);
38     mergeSort(dbls, 0, dbls.size() - 1); print(dbls);
39     mergeSort(words, 0, words.size() - 1); print(words);
40 }

```

Listing 2: Merge Sort — generic template (works on any comparable type)

Expected output:

```

1 2 3 5 8 9
0.57 1.41 2.72 3.14
apple banana cherry date

```

1.6 Key Properties of Merge Sort**Merge Sort — Properties at a Glance**

Comparisons	Always $O(n \log n)$ — same in all cases.
Extra Memory	$O(n)$ — requires a temporary buffer to hold the merged result.
Stability	Stable — the $\leq$ in the merge step preserves the relative order of equal elements.
In-place	No — uses $O(n)$ auxiliary memory.
Best for	Large datasets, linked lists, external sorting (data on disk), and whenever stability is required.

Part 2: Quick Sort**2.1 Core Idea****How Quick Sort Works**

Quick Sort is also divide-and-conquer, but it does its work *before* recursing (unlike Merge Sort, which does its work when merging *after* recursing):

1. **Choose a pivot** — select one element from the array (strategy matters; common: last element, median-of-three).

2. **Partition** — rearrange the array so that:

- Every element *less than* the pivot is to its left.
- Every element *greater than* the pivot is to its right.
- The pivot is now in its **final sorted position**.

3. **Recurse** — apply Quick Sort to the left sub-array and the right sub-array.

No merge step is needed. The partition step is $O(n)$, and with a *good pivot*, the two sub-arrays are roughly equal in size, giving $O(\log n)$ recursion levels and a total of $O(n \log n)$ work.

2.2 The Partition Step (Lomuto Scheme)

The most common partition scheme (Lomuto) works as follows using the **last element** as pivot:

i	j	Array state	Action
pivot = 10, arr = [3, 6, 8, 10, 1, 2, 1]			Initial; pivot at end
-1	0	[3, 6, 8, 10, 1, 2, 1]	arr[0]=3 ≤ 10: i++, swap(i,j) → no change
0	1	[3, 6, 8, 10, 1, 2, 1]	arr[1]=6 ≤ 10: i++, swap
1	2	[3, 6, 8, 10, 1, 2, 1]	arr[2]=8 ≤ 10: i++, swap
2	3	[3, 6, 8, 10, 1, 2, 1]	arr[3]=10 ≤ 10: i++, swap
3	4	[3, 6, 8, 10, 1, 2, 1]	arr[4]=1 ≤ 10: i++, swap
4	5	[3, 6, 8, 10, 1, 2, 1]	arr[5]=2 ≤ 10: i++, swap
Place pivot: swap(arr[i+1], pivot) ⇒ [1, 2, 3, 10, 6, 8, 10]			

Simpler example to build intuition:

Partition [10, 80, 30, 90, 40, 50, 70], pivot = 70 (last element).

After partitioning: [10, 30, 40, 50, 70, 80, 90] — pivot 70 is in its correct final position.

Left sub-array [10, 30, 40, 50] and right sub-array [80, 90] are now sorted recursively.

2.3 Step-by-Step Trace: Full Sort

Trace: sort [10, 80, 30, 90, 40, 50, 70] (pivot = last element each time)

Sub-array	Pivot	Result after partition
[10, 80, 30, 90, 40, 50, 70]	70	[10, 30, 40, 50, 70 , 80, 90]
[10, 30, 40, 50] (left)	50	[10, 30, 40, 50]
[10, 30, 40] (left)	40	[10, 30, 40]
[10, 30] (left)	30	[10, 30]
[80, 90] (right of 70)	90	[80 , 90]
Final: [10, 30, 40, 50, 70, 80, 90]		

2.4 Implementation

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Partition: place pivot in correct position
6 // Returns the final index of the pivot
7 int partition(vector<int>& arr, int low, int high) {
8     int pivot = arr[high]; // Choose last element as pivot
9     int i = low - 1;       // i tracks the boundary of the "small" region
10
11     for (int j = low; j < high; j++) {
12         if (arr[j] <= pivot) {
13             i++;
14             swap(arr[i], arr[j]); // Move small element to left region
15         }
16     }
17
18     swap(arr[i + 1], arr[high]); // Place pivot in its final position
19     return i + 1;               // Return pivot index
20 }
21
22 void quickSort(vector<int>& arr, int low, int high) {
23     if (low >= high) return; // Base case: 0 or 1 element
24
25     int pi = partition(arr, low, high); // Partition and get pivot index
26
27     cout << "Pivot " << arr[pi] << " placed at index " << pi << ": [ ";
28     for (int x : arr) cout << x << " ";
29     cout << "]\n";
30
31     quickSort(arr, low, pi - 1); // Sort left of pivot
32     quickSort(arr, pi + 1, high); // Sort right of pivot
33 }
34
35 void printArray(const string& label, const vector<int>& arr) {
36     cout << label << ": [ ";
37     for (int x : arr) cout << x << " ";
38     cout << "]\n";
39 }
40
41 int main() {
42     vector<int> data = {10, 80, 30, 90, 40, 50, 70};
43
44     printArray("Before", data);
45     cout << "\n";
46     quickSort(data, 0, data.size() - 1);
47     cout << "\n";
48     printArray("After ", data);
49 }

```

Listing 3: Quick Sort — Lomuto partition scheme with trace output

Expected output:

Before: [10 80 30 90 40 50 70]

Pivot 70 placed at index 4: [10 30 40 50 70 90 80]

Pivot 50 placed at index 3: [10 30 40 50 70 90 80]
 Pivot 40 placed at index 2: [10 30 40 50 70 90 80]
 Pivot 30 placed at index 1: [10 30 40 50 70 90 80]
 Pivot 80 placed at index 5: [10 30 40 50 70 80 90]

After : [10 30 40 50 70 80 90]

2.5 Improved Pivot: Median-of-Three

Choosing the last element as pivot is simple but performs poorly on already-sorted arrays ($O(n^2)$). A better strategy is the **median-of-three**: pick the median of the first, middle, and last elements.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 // Returns the index of the median of arr[low], arr[mid], arr[high]
7 int medianOfThree(vector<int>& arr, int low, int high) {
8     int mid = low + (high - low) / 2;
9     // Sort the three candidate values and return the middle index
10    if (arr[low] > arr[mid]) swap(arr[low], arr[mid]);
11    if (arr[low] > arr[high]) swap(arr[low], arr[high]);
12    if (arr[mid] > arr[high]) swap(arr[mid], arr[high]);
13    // arr[low] <= arr[mid] <= arr[high]; median is at mid
14    swap(arr[mid], arr[high - 1]); // Place median just before high
15    return high - 1;
16 }
17
18 int partition(vector<int>& arr, int low, int high) {
19     int pivotIdx = medianOfThree(arr, low, high);
20     int pivot = arr[pivotIdx];
21     swap(arr[pivotIdx], arr[high]); // Move pivot to end
22
23     int i = low - 1;
24     for (int j = low; j < high; j++)
25         if (arr[j] <= pivot) swap(arr[++i], arr[j]);
26
27     swap(arr[i + 1], arr[high]);
28     return i + 1;
29 }
30
31 void quickSort(vector<int>& arr, int low, int high) {
32     if (low >= high) return;
33     int pi = partition(arr, low, high);
34     quickSort(arr, low, pi - 1);
35     quickSort(arr, pi + 1, high);
36 }
37
38 int main() {
39     vector<int> data = {5, 2, 8, 1, 9, 3, 7, 4, 6};
40     quickSort(data, 0, data.size() - 1);
41
42     cout << "Sorted: ";
43     for (int x : data) cout << x << " ";
44     cout << "\n";
45 }

```

Listing 4: Quick Sort — median-of-three pivot selection

Expected output:

Sorted: 1 2 3 4 5 6 7 8 9

2.6 Key Properties of Quick Sort**Quick Sort — Properties at a Glance**

Comparisons	Best/Average: $O(n \log n)$ Worst: $O(n^2)$
Extra Memory	$O(\log n)$ stack space for recursion (in-place partitioning).
Stability	Not stable — swaps can move equal elements past each other.
In-place	Yes — no auxiliary array needed (unlike Merge Sort).
Worst case	Occurs when the pivot is always the minimum or maximum (e.g., sorted input with last-element pivot).
Best for	General-purpose in-memory sorting; fastest in practice for random data due to excellent cache behaviour.

Part 3: Time Complexity Analysis**3.1 Analysis of Merge Sort****Counting Operations in Merge Sort**

At each recursion level there are 2^k sub-arrays of size $n/2^k$. Merging all sub-arrays at one level costs $O(n)$ total comparisons. The number of levels is $\lceil \log_2 n \rceil$.

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \xrightarrow{\text{Master Theorem}} T(n) = O(n \log n)$$

Case	Comparisons	Extra Memory	Overall
Best (sorted)	$O(n \log n)$	$O(n)$	$O(n \log n)$
Average (random)	$O(n \log n)$	$O(n)$	$O(n \log n)$
Worst (reversed)	$O(n \log n)$	$O(n)$	$O(n \log n)$

Merge Sort's complexity is the same in all cases. It is **not adaptive** (it always divides perfectly in half) but it is **always** $O(n \log n)$.

3.2 Analysis of Quick Sort**Counting Operations in Quick Sort**

Best / Average case — pivot always splits array roughly in half:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) = O(n \log n)$$

Worst case — pivot is always the smallest or largest element (e.g., sorted array, last-element pivot):

$$T(n) = T(n-1) + O(n) = O(n^2)$$

With **random pivot selection** or **median-of-three**, the worst case is astronomically unlikely on random data.

Case	Comparisons	Extra Memory	Overall
Best (balanced pivots)	$O(n \log n)$	$O(\log n)$	$O(n \log n)$
Average (random)	$O(n \log n)$	$O(\log n)$	$O(n \log n)$
Worst (bad pivots)	$O(n^2)$	$O(n)$	$O(n^2)$

3.3 Empirical Comparison

```

1 #include <iostream>
2 #include <vector>
3 #include <numeric>
4 #include <algorithm>
5 using namespace std;
6
7 long long mergeComps = 0, quickComps = 0;
8
9 void merge(vector<int>& arr, int l, int m, int r) {
10     vector<int> L(arr.begin()+l, arr.begin()+m+1);
11     vector<int> R(arr.begin()+m+1, arr.begin()+r+1);
12     int i=0, j=0, k=l;
13     while (i<(int)L.size() && j<(int)R.size()) {
14         mergeComps++;
15         arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
16     }
17     while (i<(int)L.size()) arr[k++] = L[i++];
18     while (j<(int)R.size()) arr[k++] = R[j++];
19 }
20 void mergeSort(vector<int>& a, int l, int r) {
21     if (l>=r) return;
22     int m=l+(r-l)/2;
23     mergeSort(a,l,m); mergeSort(a,m+1,r);
24     merge(a,l,m,r);
25 }
26
27 int partition(vector<int>& a, int lo, int hi) {
28     int pivot=a[hi], i=lo-1;
29     for (int j=lo; j<hi; j++) { quickComps++; if (a[j]<=pivot) swap(a[++i],
30         a[j]); }
31     swap(a[i+1],a[hi]); return i+1;
32 }
33 void quickSort(vector<int>& a, int lo, int hi) {
34     if (lo>=hi) return;
35     int p=partition(a,lo,hi);
36     quickSort(a,lo,p-1); quickSort(a,p+1,hi);
37 }
38
39 void runTest(const string& label, vector<int> arr) {
40     cout << "\n--- " << label << " (n=" << arr.size() << ") ---\n";

```

```

40     mergeComps = quickComps = 0;
41     auto a2 = arr;
42     mergeSort(arr, 0, arr.size()-1);
43     quickSort(a2, 0, a2.size()-1);
44     cout << "Merge Sort: " << mergeComps << " comparisons\n";
45     cout << "Quick Sort: " << quickComps << " comparisons\n";
46 }
47
48 int main() {
49     int n = 10;
50     vector<int> sorted(n), reversed(n), random_ = {3,1,4,1,5,9,2,6,5,3};
51     iota(sorted.begin(), sorted.end(), 1);
52     iota(reversed.begin(), reversed.end(), 1);
53     reverse(reversed.begin(), reversed.end());
54
55     runTest("Already sorted", sorted);
56     runTest("Reverse sorted", reversed);
57     runTest("Random", random_);
58 }

```

Listing 5: Empirical comparison: operation counts for Merge Sort vs. Quick Sort

Expected output:

```

--- Already sorted (n=10) ---
Merge Sort: 19 comparisons
Quick Sort: 45 comparisons    <- worst case for last-element pivot!

--- Reverse sorted (n=10) ---
Merge Sort: 19 comparisons
Quick Sort: 45 comparisons

--- Random (n=10) ---
Merge Sort: 22 comparisons
Quick Sort: 21 comparisons    <- Quick Sort often beats Merge Sort in practice

```

Important observation: On sorted input with a last-element pivot, Quick Sort degrades to $O(n^2)$ (matching Selection Sort in comparisons!). Merge Sort is unaffected. This is why production implementations use randomised or median-of-three pivots.

3.4 Growth Rate Comparison

n	n^2 (Selection/Insertion worst)	$n \log_2 n$ (Merge/Quick avg)	Speed-up factor
100	10 000	664	$\times 15$
500	250 000	4 483	$\times 56$
1 000	1 000 000	9 966	$\times 100$
5 000	25 000 000	61 439	$\times 407$
10 000	100 000 000	132 877	$\times 753$

3.5 Full Side-by-Side Comparison

Merge Sort vs. Quick Sort — Full Comparison

Property	Merge Sort	Quick Sort
Best-case time	$O(n \log n)$	$O(n \log n)$
Average-case time	$O(n \log n)$	$O(n \log n)$
Worst-case time	$O(n \log n)$	$O(n^2)$
Space complexity	$O(n)$ auxiliary	$O(\log n)$ stack
Stable	Yes	No (standard)
Adaptive	No	Partially (bad pivots matter)
In-place	No	Yes
Cache performance	Good	Excellent (in-place)
Preferred use	Stability required, linked lists, external sort	General in-memory; fastest in practice

3.6 Practical Guidelines

When to Use Each Algorithm

Use Merge Sort when:

- A **stable** sort is required (equal elements must preserve original order).
- You are sorting a **linked list** (no random access, no cache advantage for Quick Sort).
- You are performing **external sorting** (data too large for RAM; merging is sequential-access friendly).
- A guaranteed $O(n \log n)$ worst case is required (real-time systems).

Use Quick Sort when:

- You need the **fastest average-case performance** in memory.
- Memory is limited (Quick Sort needs only $O(\log n)$ stack space vs. $O(n)$ for Merge Sort).
- You use a **good pivot strategy** (median-of-three or randomised) to avoid $O(n^2)$.
- It is the default in many standard libraries (`std::sort` uses Introsort, which falls back to Heap Sort if recursion goes too deep).

Hybrid approaches used in practice:

- **Introsort** (C++ `std::sort`) — Quick Sort + Heap Sort fallback + Insertion Sort for small n .
- **Timsort** (Python `sorted()`, Java `Arrays.sort`) — Merge Sort + Insertion Sort for small runs; exploits existing order.
- Both switch to Insertion Sort for sub-arrays of size ≤ 16 because constant factors dominate at small n .

Lab Exercises

Exercise 1

[Merge Sort — Trace] Given the array `[38, 27, 43, 3, 9]`, manually draw the full recursion tree showing every divide step and every merge step. For each merge, write down the two input sub-arrays and the resulting merged array. Then implement the sort, run it, and verify your trace.

Exercise 2

[Merge Sort — Counting Comparisons] Modify the `merge` function to count comparisons. Run it on: (i) an already-sorted array, (ii) a reverse-sorted array, and (iii) a random array, all of size $n = 16$. Show that the comparison counts are nearly identical in all three cases and match the theoretical $\Theta(n \log n)$ bound.

Exercise 3

[Merge Sort — Stability Test] Define `struct Card { int value; char suit; }`. Verify that Merge Sort preserves the relative order of cards with equal values (i.e., it is stable). Then confirm that Quick Sort (Lomuto) does *not* guarantee stability on the same input.

Exercise 4

[Quick Sort — Partition Trace] Given the array `[8, 3, 7, 1, 5, 2, 4]`, manually perform the Lomuto partition using `arr[6] = 4` as the pivot. Write the state of the array after every swap, and identify the final position of the pivot. Then implement and verify.

Exercise 5

[Quick Sort — Worst Case] Demonstrate the $O(n^2)$ worst case:

- Create an already-sorted array of size $n = 15$ and run Quick Sort with the last-element pivot.
- Count comparisons. Compare with the theoretical $n(n-1)/2$.
- Now switch to a *random pivot* and repeat. Report how the comparison count changes.

Exercise 6

[Counting Operations — Both Sorts] Run both Merge Sort and Quick Sort on the three inputs of size $n = 15$ below. For each run, count and print comparisons. Fill in the table and explain whether empirical results match theory.

Input	Merge Sort comparisons	Quick Sort comparisons
Sorted (best)		
Reversed (worst)		
Random (average)		

Exercise 7

[Generic Merge Sort with Custom Comparator] Write a generic Merge Sort:

```
template <typename T, typename Compare>
void mergeSort(vector<T>& arr, int left, int right, Compare cmp);
```

Use it to sort: (i) `vector<int>` descending; (ii) `vector<string>` by length ascending; (iii) a `vector<pair<int,string>>` by the integer key, breaking ties alphabetically.

Exercise 8

[**Sorting Structs**] Define `struct Student { string name; double gpa; int year; }`. Implement both Merge Sort and Quick Sort that sort a `vector<Student>` by: (i) GPA descending; (ii) year ascending, then GPA descending; (iii) name alphabetically ascending. Verify that Merge Sort's result is stable (students with equal keys preserve original order).

Exercise 9

[**Hybrid Sort: Quick + Insertion**] Implement a hybrid sort that:

- Uses **Insertion Sort** when the sub-array size $\leq$ threshold k .
- Uses **Quick Sort** (with median-of-three pivot) otherwise.

Test on $n \in \{50, 200, 1000\}$ with $k \in \{5, 10, 20\}$. Print comparison counts for every combination and identify the threshold that minimises comparisons. Explain why small sub-arrays benefit from Insertion Sort.

Exercise 10

[**Performance Benchmark: All Four Sorts**] Write a program that runs all four sorts (Selection, Insertion, Merge, Quick) on the same random input of size $n \in \{100, 1000, 10000\}$. For each sort and each n , record wall-clock time using `<chrono>`. Print a table and comment on the relative performance. At what value of n does the $O(n \log n)$ advantage become clearly visible?
